JOHANNES KEPLER
UNIVERSITY LINZ

Author
Tim Michael Toller
k11842616

Submission
Institute for Application-oriented Knowledge Processing (FAW)

Thesis Supervisor
A.Univ.Prof., DI Dr.
Josef Küng

January 24, 2026

Watermarks for Language Models



Practical work in AI — Documentation

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

1. Introduction **3**
1.1 Motivation and Problem Statement . . . 3
1.2 Project Goals and Scope . . . 3

2. Background: LLM Watermarking and Exp-Min **4**
2.1 What Text Watermarking Is . . . 4
2.2 Exp-Min Watermarking: Idea and Properties . . . 4
2.3 Detection Principle: Test Statistic and p-value Interpretation . . . 5

3. Backend **5**
3.1 Project Structure and Entry Points . . . 5
3.2 Model Backend Integration . . . 5
3.3 API Design . . . 6
3.3.1 Health and Model Info (`/health`, `/model`) . . . 6
3.3.2 Generation Endpoints (`/generate`, `/generate/stream`) . . . 6
3.3.3 Detection Endpoints (`/detect`, `/detect/stream`) . . . 6
3.3.4 Corruption Endpoint (`/corrupt`) . . . 6
3.4 Configuration and Runtime Settings . . . 7

4. Watermarking and Detection Core **7**
4.1 Watermark Parameters and their roles . . . 7
4.2 Generation Algorithm . . . 8
4.3 Detection Algorithm . . . 8

5. Frontend Implementation **9**
5.1 Structure and Architecture . . . 9
5.2 State Management . . . 9
5.3 Backend Integration and Streaming . . . 9
5.4 Generation . . . 9
5.5 Corruption (Adversarial Editing) . . . 10
5.6 Detection . . . 10
5.7 Screenshots . . . 11

6. Setup and Usage **12**
6.1 Requirements and Installation . . . 12
6.2 Running as a Docker Compose Container . . . 12
6.3 Running for Development . . . 12

7. Conclusion **13**
7.1 Summary of Results . . . 13
7.2 Limitations . . . 13
7.3 Reproducibility and Practical Use . . . 13
7.4 Future Work and Outlook . . . 14

References **15**

1. Introduction

1.1 Motivation and Problem Statement

Large Language Models are increasingly capable of generating fluent, coherent, and contextually appropriate text at scale. While this progress enables a wide range of beneficial applications, it also raises significant concerns regarding the provenance and authenticity of generated content. In academic, journalistic, and online community contexts, it is often crucial to determine whether a piece of text was authored by a human or produced by an automated system. The inability to reliably make this distinction can lead to challenges in academic integrity, plagiarism detection, copyright enforcement, and the propagation of misinformation. It also makes further training of new models on AI-generated content more likely as more content online is AI-generated, which can lead to model degradation over time.

Several platforms have already introduced explicit policies governing the use of AI-generated content [2]. However, policy-based approaches alone are difficult to enforce, as generated text can easily be copied, edited, or redistributed without attribution. This creates a technical problem: how can the origin of text generated by a language model be verified once it has left the control of the original provider?

Text watermarking offers a potential solution to this problem. By embedding a hidden statistical signal into generated text at generation time, it becomes possible to later test whether a given text was likely produced by a specific language model using a particular secret key. Ideally, such a watermark should be invisible to users, robust against common forms of text modification, and reliably detectable with a well-defined statistical guarantee.

Despite recent progress in this area, many watermarking approaches remain either theoretical or limited to offline experiments. Bridging the gap between theory and practical deployment remains an open challenge, especially when considering performance constraints, real-time generation, and interactive user interfaces.

1.2 Project Goals and Scope

The goal of this practical work is to design and implement a *practical, end-to-end demonstration* of a modern text watermarking scheme for language models. The implementation is inspired primarily by the work of Kuditipudi et al. [1], who propose a robust, distortion-free watermarking method based on Exponential-Minimum (Exp-Min) sampling.

Building on earlier seminar work, which focused on a Jupyter notebook-based proof of concept, this project extends the ideas into a deployable system consisting of:

- A Python FastAPI backend responsible for text generation, watermark embedding, watermark detection, and controlled text corruption.
- A React-based frontend that allows users to interactively generate text, apply corruption strategies, and observe detection results in real time in a user-friendly manner.
- A streamlined watermarking and detection core that preserves the theoretical foundations of the original method while introducing selected modifications to improve runtime performance, usability for live demonstrations and deployment.

The scope of the project is intentionally practical rather than purely theoretical. While the watermarking algorithms are grounded in existing research, the focus lies on system

design, API structure, and real-world constraints such as latency, streaming generation, and visualization of detection results. This project should not act as a one-to-one reproduction of the original paper, where deviations are introduced, these changes are explicitly discussed and justified.

2. Background: LLM Watermarking and Exp-Min

2.1 What Text Watermarking Is

Text watermarking is the process of embedding a hidden signal into generated text such that the signal can later be detected to infer the text's origin. In the context of language models, watermarking is typically performed during token generation rather than as a post-processing step.

A standard watermarking protocol involves three conceptual roles:

1. **Language Model Provider**: Generates text using a language model and embeds a watermark using a secret key.
2. **User**: Receives the generated text and may publish it, modify it, or attempt to remove the watermark.
3. **Detector**: Given a piece of text and the secret key, determines whether the text is likely to contain the watermark.

Crucially, the detector does not require access to the original prompt or the language model itself. Instead, detection is performed by evaluating a statistical test on the observed token sequence with respect to the secret key. The user is treated as a potential adversary who may attempt to evade detection through paraphrasing, editing, or partial deletion of the text.

2.2 Exp-Min Watermarking: Idea and Properties

A central challenge in watermarking language model outputs is preserving output quality. There are many approaches to watermarking, but naïvely biasing token probabilities can introduce noticeable artifacts or degrade fluency. Kuditipudi et al. [1] address this by proposing *Exponential-Minimum (Exp-Min) sampling*, a method that should preserve the original model distribution exactly.

At each generation step, the language model produces a probability distribution

$$p : V \to [0, 1]$$

over the vocabulary V. A secret watermark key is used to generate a corresponding vector of independent random variables

$$\omega \sim \mathcal{U}(0, 1)^{|V|}.$$

The next token t^* is selected as

$$t^* = \arg\min_{t \in V} \frac{-\ln \omega_t}{p(t)}.$$

This procedure samples tokens as if they were drawn from the original distribution p, ensuring distortion-free generation. However, across many tokens, a subtle correlation is introduced between the chosen tokens and the secret key. This correlation constitutes the watermark.

2.3 Detection Principle: Test Statistic and p-value Interpretation

Watermark detection is framed as a hypothesis testing problem. Given a text sequence, the detector evaluates whether the observed token sequence is more consistent with the watermark key than would be expected under random keys.

Detection proceeds by:

1. Computing a test statistic that measures alignment between the observed tokens and the watermark key.
2. Comparing this statistic against a null distribution obtained from randomly generated surrogate keys.
3. Computing a p-value representing the probability that an unwatermarked text would produce a statistic at least as extreme as the observed one.

If the p-value falls below a predefined threshold (the suggested threshold by Kuditipudi et al. is $p < 0.01$), the text is considered watermarked.

To improve robustness against edits, insertions, and deletions, the detection process can incorporate a Levenshtein-style alignment cost. This allows the detector to tolerate partial mismatches between the generated watermark sequence and the observed text, making the watermark resilient to paraphrasing and moderate corruption.

3. Backend

3.1 Project Structure and Entry Points

The backend is implemented in Python using `FastAPI`. The main entry point is the FastAPI application in `backend/app/main.py`, which registers all route modules and configures CORS for the local development frontend.

Dependency management is handled via `uv`, which provides fast, reproducible installs and an ergonomic way to run the application (see also the `backend/Dockerfile` and the project `README.md`).

At a high level, the backend code is organized as follows:

- `backend/app/`: FastAPI application, request/response schemas, and HTTP routes.
- `backend/app/routes/`: Route modules
- `backend/app/schemas.py`: Pydantic models for request and response payloads.
- `backend/core/`: Core watermarking logic
- `backend/model/`: Abstraction over the model backend and the `llama-cpp-python` implementation.
- `backend/models/`: Local model files (`.gguf`) used by `llama-cpp-python`.

3.2 Model Backend Integration

Compared to the original implementation by Kuditipudi et al.[1], which loads models through the `transformers` stack, this project uses `llama-cpp-python` to run quantized `.gguf` models efficiently on consumer hardware. This change resulted in many changes

in the code of the token generation process, but also in faster inference times and lower memory usage.

The backend interface is defined in `backend/model/backend.py`, and the concrete implementation is `backend/model/llama_cpp_backend.py`. This backend is instantiated via FastAPI dependency injection in `backend/app/deps.py` and cached using `lru_cache` so the model is loaded only once per process.

In principle, any `.gguf` model compatible with `llama-cpp-python` can be used. For the practical work we used a small instruction-tuned model (Llama 3.2 3B Instruct) because it offers a good speed/quality trade-off. However, instruction tuning can generate text with low watermark potential, so this has to be kept in mind. Models with open-ended text generation capabilities are likely to yield better watermarking results.[1]

3.3 API Design

3.3.1 Health and Model Info (`/health`, `/model`)

`/health` is a minimal liveness endpoint. It returns a small JSON object (`{"ok": true}`) to indicate that the API is up.

`/model` returns basic runtime configuration and model metadata used by the frontend and for debugging. In particular, it exposes the configured context size and threading/GPU settings as well as the resolved model path and vocabulary size.

3.3.2 Generation Endpoints (`/generate`, `/generate/stream`)

Both generation and detection are provided as a standard non-streaming and a streaming endpoint. The frontend uses the streaming variants for a more responsive user experience, while the non-streaming endpoints are convenient for manual testing.

`/generate` returns the final generation result as a single JSON payload.

`/generate/stream` streams results via Server-Sent Events (SSE, `text/event-stream`). The stream sends incremental `token` events as each new token is sampled, and finishes with a `final` event containing the full `GenerateResponse` payload.

3.3.3 Detection Endpoints (`/detect`, `/detect/stream`)

`/detect` tokenizes the provided text and runs the watermark detector. The response includes an estimated p-value and a coarse interpretation (`unlikely`/`likely`/`very likely`) based on fixed thresholds.

`/detect/stream` also uses SSE. It emits an initial `init` event with the chosen parameters, periodic `progress` events during the Monte Carlo comparison, and a terminating `final` event. For speed, the implementation includes a simple early-stopping heuristic: if intermediate bounds imply that the final classification cannot change, the endpoint stops the Monte Carlo loop early and reports the reason.

3.3.4 Corruption Endpoint (`/corrupt`)

To evaluate robustness against simple corruption attacks, `/corrupt` produces a perturbed version of an input text.

The original paper evaluates corruption using back-translation. This project initially explored that idea, but the translation toolchain significantly increased dependencies and latency. For a lightweight and fast alternative, the current endpoint applies randomized word-level mutations (swap/drop/duplicate/replace characters) until the output roughly reaches a requested similarity target. This method isn't as linguistically informed as back-translation, but it serves as a very fast proxy for simple adversarial editing.

The request parameter is `similarity_goal` in $[0, 1]$: values closer to 1.0 apply little to no corruption, while lower values increase the amount of perturbation. Similarity is measured using `difflib.SequenceMatcher`, and the response reports both the achieved similarity and the deviation from the target.

3.4 Configuration and Runtime Settings

Runtime configuration is defined in `backend/core/settings.py` using `pydantic-settings`. All settings can be overridden via environment variables (case-insensitive).

Default model settings:

```
model_path: "./backend/models/Llama-3.2-3B-Instruct-Q4_K_M.gguf"
n_ctx: 4096
n_threads: 0        # 0 -> auto CPU count
n_gpu_layers: -1
top_n_probs: 512
```

4. Watermarking and Detection Core

4.1 Watermark Parameters and their roles

This prototype implements an "exp-min" watermark for autoregressive language model generation together with a statistical detector. Both algorithms are keyed: a secret integer key deterministically generates a pseudo-random table ξ of values in $[0, 1)$ that is used during both generation and detection.

In the backend API, watermarking and detection are controlled by the following parameters:

- **Key (`key`).** Secret seed that defines the watermark. The generator and detector derive the same pseudo-random values from this key; using the wrong key makes the detector behave like it is under the null hypothesis.
- **Sequence length (`n`).** The watermark has a periodic structure of length n. At generation step t, the algorithm uses a row index $r = (\tau + t) \bmod n$; during detection the statistic is minimized over possible cyclic offsets to account for the unknown shift τ.
- **Generated length (`m`).** Number of new tokens produced by the watermarked sampler. (This is distinct from n, which controls watermark periodicity.)
- **Detection window (`k`).** Number of tokens from the analyzed text used in detection (defaults to m). Larger k generally increases detection power but costs more computation.

- **Edit penalty (γ).** Non-negative penalty used by the detector to allow insertions/deletions in a Levenshtein-style dynamic program. Intuitively, γ controls how tolerant detection is to corruptions such as added/removed tokens.
- **Monte Carlo runs (`n_runs`).** Number of random trials used to estimate a p-value under the null. More runs yield a more stable p-value estimate but increase runtime.
- **Sampling controls (temperature/top-p).** Applied to the model logits before watermarked sampling. These affect generation quality and diversity; the watermarking step is applied after these transformations.

4.2 Generation Algorithm

The generation endpoint implements explicit token-by-token decoding using the model backend's full next-token logits. The algorithm can be summarized as follows:

1. **Initialize context.** The prompt is tokenized and evaluated once by the model backend to obtain the next-token logits.
2. **Row schedule.** A key-derived shift $\tau \in \{0, \ldots, n-1\}$ is computed, and the watermark row used at step t is $r = (\tau + t) \bmod n$.
3. **Compute sampling distribution.** The current logits are converted into probabilities p after applying temperature and optional nucleus (top-p) filtering.
4. **Keyed "randomness" over the vocabulary.** For the current row r, a deterministic vector $u \in [0,1)^V$ is generated over the full vocabulary (where V is the vocabulary size). This vector is derived from the secret key and the row index.
5. **Exp-min choice.** The next token is selected by minimizing an exp-min score

$$i^* = \arg\min_i \frac{-\log(u_i)}{\max(p_i, \varepsilon)}$$

   where ε is a small constant for numerical stability.
6. **Advance.** The chosen token is appended to the context, evaluated by the model to obtain logits for the next step, and the process repeats for m steps.

4.3 Detection Algorithm

Detection tests whether a given text is consistent with having been produced by the keyed exp-min sampler.

First, the input text is tokenized and only the first k tokens are used (by default, k is set to the full input length but should ideally be capped for very large inputs). Let the resulting token sequence be $x_1, \ldots, x_k$.

The detector constructs a keyed pseudo-random matrix $\xi \in [0,1)^{n \times U}$, where U is the number of unique tokens among the first k tokens. Each entry $\xi_{r,u}$ is deterministically derived from the secret key, the row index $r \in \{0, \ldots, n-1\}$, and the token id.

From this it derives costs $w_{r,u} = \log(1 - \xi_{r,u})$ (with clipping to avoid $\log(0)$). The core detection statistic is then computed using a Levenshtein-style dynamic program that aligns the observed token sequence against the periodic watermark row sequence. Insertions and deletions cost γ, while diagonal (match) transitions use the keyed costs

$w_{r,u}$. Because the true shift τ is unknown, the detector minimizes this alignment cost over a set of candidate cyclic offsets.

To turn the statistic into a p-value, the backend uses a Monte Carlo procedure: it repeatedly samples an i.i.d. uniform "null" matrix ξ' of the same shape, recomputes the statistic, and counts how often the null statistic is at least as extreme as the observed one. The resulting estimate is reported as a smoothed p-value, and the implementation may early-stop when bounds imply the outcome is certainly "unlikely" ($p > 0.05$), "likely" ($0.01 < p \leq 0.05$), or "very likely" ($p \leq 0.01$).

5. Frontend Implementation

5.1 Structure and Architecture

The frontend is implemented as a single-page application using React (TypeScript) and built with Vite. The package manager used is `pnpm`, a fast and disk-space-efficient alternative to `npm` and `yarn`. The UI is composed using `shadcn/ui` primitives (cards, tabs, inputs, accordions, badges) and styled with Tailwind CSS. Icons are provided via `lucide-react` and small loading/streaming animations are implemented using `motion`. For visualization, the detection view uses `recharts` to render the running $\hat{p}$ curve.

The application layout is intentionally compact: the main panel contains two tabs (*Generate* and *Detect*) and a second column shows a small *Model info* sidebar. This sidebar queries the backend once on page load to display runtime parameters.

5.2 State Management

State is handled with Jotai. User-facing settings (active tab, generation settings, and detection settings) are stored using `atomWithStorage`, which persists them in the browser (`localStorage`) across reloads. This makes it easier to iterate on experiments without repeatedly re-entering parameters such as the watermark key or n.

5.3 Backend Integration and Streaming

All API calls are performed using `fetch`. The API base URL is configurable via `VITE_API_BASE_URL`. If it is not set, requests default to `/api`. Vite proxies `/api/*` to the backend at `http://localhost:8000` and rewrites the path (see Vite proxy configuration), which avoids CORS issues.

Generation and detection are implemented as server-sent event (SSE) streams. Because both endpoints use HTTP `POST` (to send JSON settings), the frontend reads the response via a `ReadableStream` and parses SSE frames manually. The stream is then mapped to UI updates (token-by-token output for generation, and incremental $\hat{p}$ values for detection).

5.4 Generation

In the *Generate* tab, the user provides a prompt in a text area. For convenience, a shuffle button loads a random prompt from a bundled `prompts.json` file. The user can configure the most relevant parameters:

- maximum tokens m,
- watermark dimension n,
- watermark key,
- sampling temperature,
- and an optional *vanilla baseline* output (non-watermarked text) for comparison.

When generation starts, the frontend calls the backend streaming endpoint and renders tokens as they arrive. The final event contains the completed text and timing information (and optionally the vanilla baseline text). From this view the result can be copied to the clipboard or sent directly to the *Detect* tab.

5.5 Corruption (Adversarial Editing)

Below the generation output, a dedicated *Corrupt Text* card supports controlled editing of the generated text. The text area is editable, enabling manual adversarial changes. Additionally, a *Corrupt* button calls the backend corruption endpoint with a fixed similarity target of 0.9 (i.e., aiming for roughly 10% distortion).

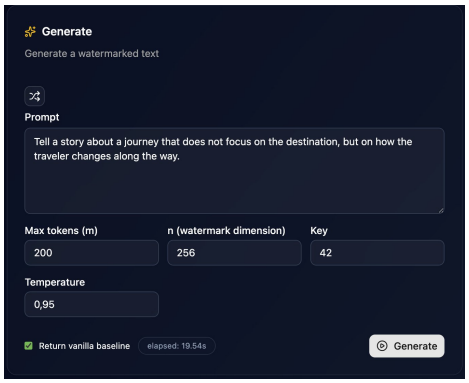
To provide immediate feedback, the UI computes a character-level similarity score based on normalized Levenshtein distance and displays it above the editor. An optional inline diff preview highlights insertions and deletions, making it easy to understand how the corruption changed the text. The corrupted text can then be copied or sent directly to the detector.

5.6 Detection

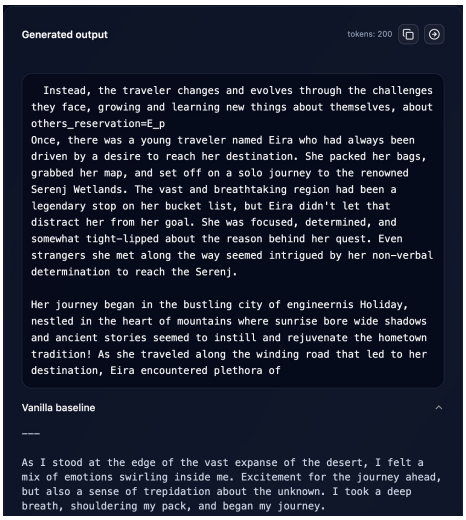
In the *Detect* tab, the user pastes (or forwards) a text and configures the detector settings to match the generation configuration. The core parameters are n, the key, and the number of null runs.

Detection also runs as an SSE stream. The frontend collects intermediate *comparing* progress events, each contributing a new point to the running $\hat{p}$ curve. Once finished, the final event provides the estimated p-value and a qualitative label (*very likely*, *likely*, or *unlikely* watermarked). The chart visualizes the running $\hat{p}$ alongside fixed threshold lines (0.01 and 0.05), and the curve colour changes based on the current value.

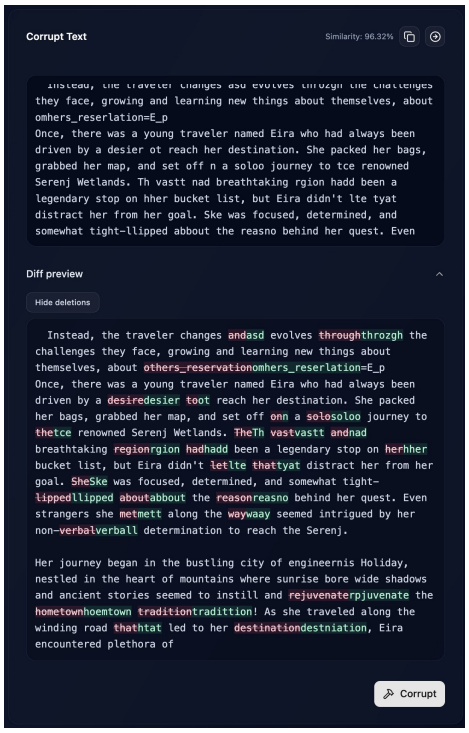
5.7 Screenshots



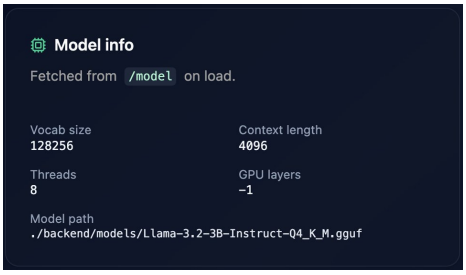
(a) Generation input



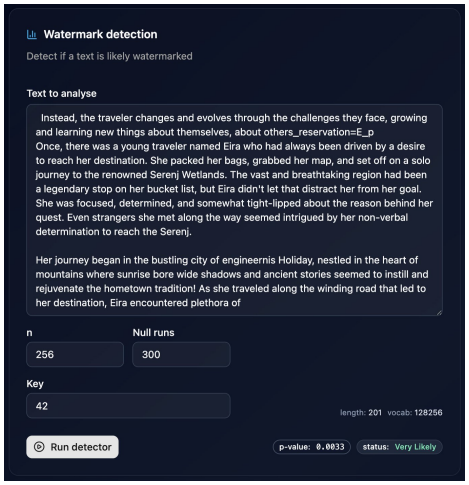
(b) Generation output (with vanilla output)



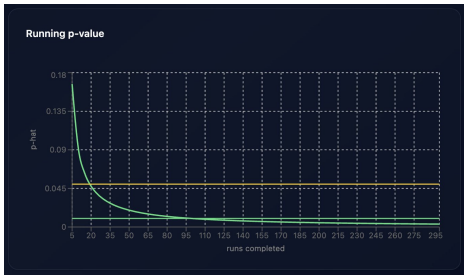
(c) Corruption interface



(d) Model info sidebar



(e) Detection input



(f) Detection output with running $\hat{p}$ curve

6. Setup and Usage

6.1 Requirements and Installation

There are two main ways to run the project:

1. using Docker Compose to start the full stack as containers
2. starting the backend and frontend separately for development

More detailed, step-by-step instructions are provided in `README.md`.

For the recommended setup, we use Docker and Docker Compose because it provides easier deployment and consistent dependency management across different machines (no local Python/Node toolchains are required).

In addition to the container runtime, a local GGUF model file is required. The default configuration expects the model to be available at: `backend/models/llama-3.2-3b-instruct-q4_k_m.gguf`. If a different filename is used, the environment variable `MODEL_PATH` must be updated accordingly.

Prerequisites:

- Docker and Docker Compose
- A downloaded GGUF model placed in `backend/models/`

6.2 Running as a Docker Compose Container

The production-like setup can be started with: `docker compose up -build`. This launches both services and configures the frontend to proxy API requests under `/api` to the backend. The default ports are `8080` for the frontend and `8000` for the backend API.

6.3 Running for Development

For local development without containers, the backend can be run with Python (via `uv`) and the frontend with Node.js (via `pnpm`). This mode is useful for rapid iteration (hot reload), but requires the corresponding toolchains to be installed locally.

7. Conclusion

7.1 Summary of Results

The main goal of this project was to build an accessible, interactive demonstration of LLM watermarking that is both responsive on consumer hardware and faithful to the core ideas proposed in the reference approach. We achieved this by implementing a complete, end-to-end workflow with watermarking, detection, and corruption/adversarial editing and exposing it through a simple user interface.

Across typical prompts, the system produces outputs that remain usable and natural while still allowing reliable detection of the watermark. After adversarial editing, the watermark remains measurable under moderate corruption, illustrating the intended robustness-distortion trade-off. Importantly, the prototype also demonstrates that these concepts can be explored even with a comparatively small, locally runnable model, which makes watermarking experimentation more approachable and reproducible.

Overall, the implementation supports the central takeaway: watermarking can be a practical mechanism for signaling likely model-generated text, but it should be treated as a probabilistic indicator rather than a cryptographic proof.

7.2 Limitations

The system still exhibits failure modes in certain cases. We observed situations where the watermark either introduces more distortion than expected or becomes harder to detect even without explicit adversarial editing. These outcomes can depend on multiple interacting factors, including prompt style (open-ended prompts generally perform better), sampling settings, model size/quality, and implementation details that may diverge subtly from the original method.

There are also practical constraints in performance and model choice. Development and testing were primarily done on a lightweight local machine, which limits feasible model sizes and slows inference, especially when running inside containers. As a result, the prototype prioritizes a smooth demo experience over exhaustive evaluation across many model families, decoding strategies, languages, and long-context scenarios.

Finally, the threat model is necessarily limited: determined attackers can apply stronger transformations (paraphrasing, translation, style transfer, or model-based rewriting) that may remove or significantly weaken the watermark. This reinforces that watermarking is best viewed as one component in a broader provenance and misuse-mitigation toolbox.

7.3 Reproducibility and Practical Use

To make the work easier to validate and extend, the system is structured as a self-contained stack with a clear separation between the watermarking core, model backend, and the user-facing interface. The most effective settings found during experimentation are provided as defaults so the demo behaves sensibly out of the box.

In practical deployments, users should explicitly document the assumed threat model, the chosen decoding parameters, and the detection thresholds, since these choices materially affect both distortion and detectability.

7.4 Future Work and Outlook

Several extensions would strengthen both scientific validity and real-world relevance. First, a systematic benchmark suite should be added: standardized prompt sets, multiple languages, long-form generations, and a wider range of corruption operators. Second, the implementation should be evaluated across additional model families and scales to quantify how model quality and sampling strategies affect watermark strength.

With these improvements, the project can evolve from a convincing demonstration into a stronger experimental platform for studying watermarking under realistic conditions.

References

[1] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. 2023. Robust Distortion-free Watermarks for Language Models. *arXiv preprint arXiv:2307.15593*. arXiv: 2307.15593 [cs.LG]. https://crfm.stanford.edu/2023/07/30/watermarking.html.

[2] Stack Overflow. 2022. Temporary policy: Generative AI on Stack Overflow. Retrieved 04/26/2025 from https://stackoverflow.com/help/gen-ai-policy.